

Protección de Sistemas ICS/SCADA en Infraestructuras Críticas

Caso de estudio: AquaControl S.A. — Planta de Tratamiento de Agua

Alejandro Moreno Ruz — Máster en Ciberseguridad

Tutores: Prof. Román Ramírez · Javier Domínguez — Septiembre 2026

Índice

1. Introducción
2. Descripción del escenario
 - 2.1 La empresa ficticia
 - 2.2 Arquitectura con deficiencias intencionadas
3. Marco normativo
 - 3.1 IEC 62443 — zonas y conductos
 - 3.2 NIS2 — obligaciones del sector agua
 - 3.3 ENS y CNPIC
4. Análisis de amenazas — MITRE ATT&CK for ICS
5. Laboratorio y stack tecnológico
 - 5.1 Arquitectura del despliegue
 - 5.2 Incidencias de reproducibilidad resueltas
 - 5.3 Verificación del stack
6. Demostración de ataques
 - 6.1 Reconocimiento de la red OT (T0846)
 - 6.2 Identificación de registros (T0861)
 - 6.3 Escritura no autorizada (T0855)
 - 6.4 Análisis de impacto
7. Medidas de protección
 - 7.1 Detección: sensor Modbus y reglas Wazuh
 - 7.2 Corrección: segmentación real y conducto controlado
8. Conclusiones
9. Bibliografía

1. Introducción

El presente Trabajo de Fin de Máster aborda la protección de sistemas ICS/SCADA e infraestructuras críticas. La elección responde a tres motivos principales.

En primer lugar, la relevancia geopolítica actual de los ataques contra infraestructuras críticas: los sistemas ICS/SCADA son objetivo prioritario en el contexto de las tensiones geopolíticas recientes, con antecedentes de campañas dirigidas contra infraestructuras de energía y agua. En segundo lugar, esta opción permite integrar de forma natural varias disciplinas de la ciberseguridad: protección de infraestructuras críticas, hacking hardware, ingeniería inversa de protocolos industriales, criptografía aplicada a comunicaciones OT y arquitectura de ciberseguridad. En tercer lugar, el laboratorio propuesto es completamente reproducible mediante contenedores Docker, de forma que cualquier persona puede desplegar el entorno completo en un ordenador distinto al del autor.

El objetivo del TFM es doble: (1) documentar y demostrar de forma práctica una arquitectura IT/OT deliberadamente imperfecta, evitando el escenario irreal de una empresa perfectamente protegida, y (2) proponer, implementar y **verificar empíricamente** las medidas de corrección correspondientes, en lugar de limitarse a enunciarlas de forma teórica.

El escenario elegido es una planta municipal de tratamiento de agua, **AquaControl S.A.**, cuya red industrial se conectó a la red corporativa sin las medidas de segmentación adecuadas. Sobre este escenario se ejecutan tres fases: documentación de las deficiencias, demostración de un ataque real contra el protocolo Modbus, y despliegue de las medidas de detección y corrección correspondientes, verificadas en un segundo laboratorio independiente.

Todo el código, la configuración de despliegue y los registros de laboratorio de este TFM están disponibles públicamente en <https://github.com/alejandromorenoruz/tfm-aquacontrol>.

2. Descripción del escenario

2.1 La empresa ficticia

AquaControl S.A. es una empresa ficticia del sector de infraestructuras críticas (suministro de agua potable) con las siguientes características:

- **Sector:** infraestructura crítica, suministro de agua potable a una ciudad de tamaño medio.
- **Tamaño:** 200 empleados, 3 plantas distribuidas geográficamente y un centro de control remoto.
- **Contexto:** la red OT se conectó a la red IT corporativa hace 5 años para reducir costes de mantenimiento remoto, sin implantar las medidas de segmentación adecuadas.

2.2 Arquitectura con deficiencias intencionadas

El escenario se ha diseñado de forma deliberadamente imperfecta —una arquitectura ideal no sería representativa de un caso real— por lo que se definieron las siguientes deficiencias de diseño como punto de partida del análisis:

- **Sin DMZ entre IT y OT:** la red de oficinas y la red industrial comparten conectividad directa.

- **Modbus TCP expuesto sin autenticación:** el controlador que gestiona las válvulas y bombas acepta comandos de cualquier origen de la red sin verificar identidad.
- **HMI con credenciales por defecto:** la interfaz de supervisión usa usuario/contraseña de fábrica (admin/admin), nunca cambiada.
- **Sin monitorización OT:** no existe, en el estado inicial, ningún sistema de detección de anomalías en la red industrial.
- **Firmware sin actualizar:** los controladores llevan varios años sin actualizaciones.

La figura 1 muestra el diagrama de red correspondiente a este estado deficiente.

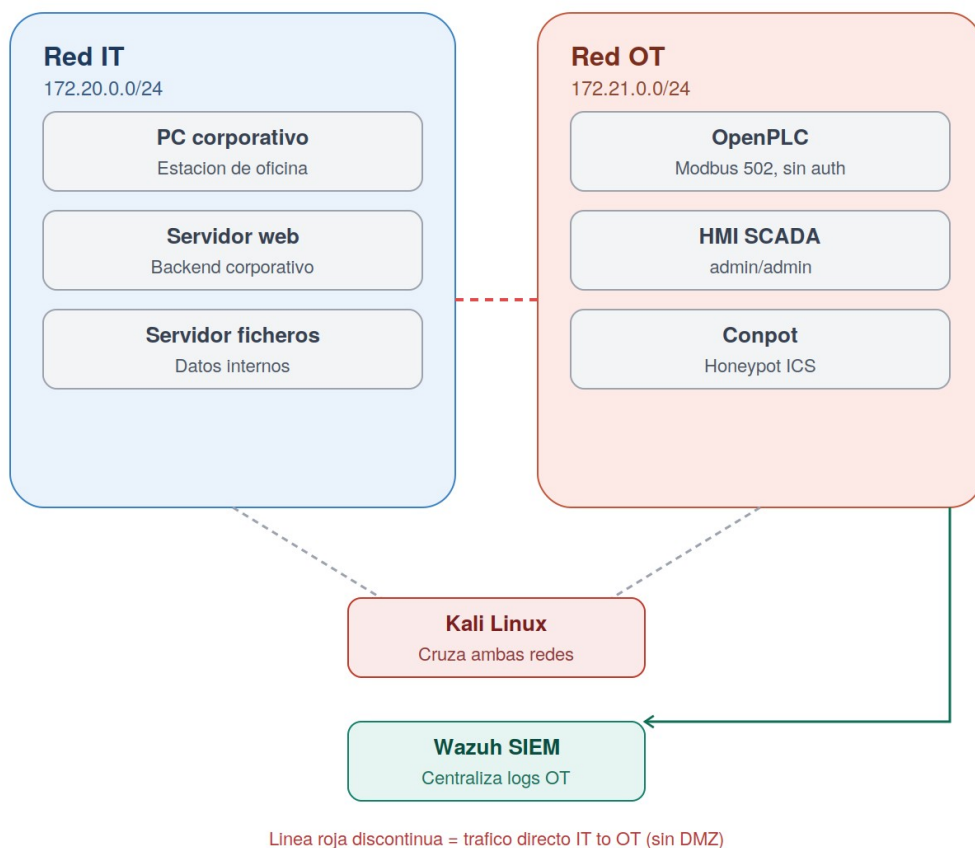


Figura 1. Arquitectura de red deficiente de AquaControl S.A.

Dentro del laboratorio, esta arquitectura se modela con dos redes Docker independientes (`red_it`, 172.20.0.0/24, y `red_ot`, 172.21.0.0/24) y un atacante (`kali`) conectado simultáneamente a ambas, representando un puesto de la red IT ya comprometido con acceso directo a la red industrial — la deficiencia central que se explota en la sección 6.

3. Marco normativo

3.1 IEC 62443 — zonas y conductos

IEC 62443 es el estándar internacional de ciberseguridad para sistemas de automatización y control industrial.¹ Su concepto central es la división de la red en **zonas de seguridad** conectadas mediante **conductos controlados**:

- **Zona 0 — Proceso físico:** sensores, actuadores, válvulas, sin conectividad IP directa.
- **Zona 1 — Control básico:** controladores lógicos (OpenPLC en el laboratorio), que solo debe aceptar comandos desde la Zona 2.
- **Zona 2 — Supervisión:** interfaz HMI y sistemas de histórico.
- **Zona 3 — Corporativa (IT):** red de oficinas, sin acceso directo a las zonas 0-2.
- **Conducto IT-OT:** único punto de comunicación autorizado entre la Zona 3 y la Zona 2.

En el escenario inicial no existe ninguna de estas zonas. La sección 7 detalla cómo se implementó y verificó esta segmentación.

3.2 NIS2 — obligaciones del sector agua

La Directiva (UE) 2022/2555 (NIS2) clasifica el suministro de agua como infraestructura crítica.² AquaControl, como operador de este sector, estaría obligada a implantar medidas técnicas y organizativas de gestión de riesgos (Art. 21), notificar incidentes significativos en un plazo de 24-72 horas, y garantizar la seguridad de su cadena de suministro tecnológico. El diseño de detección y segmentación implementado en este TFM (secciones 6 y 7) da respuesta directa a estas obligaciones.

3.3 ENS y CNPIC

El **Esquema Nacional de Seguridad** (RD 311/2022) aplicaría si AquaControl prestase servicio a una administración pública (suministro municipal). El **CNPIC**, dependiente del Ministerio del Interior, publica el Catálogo Nacional de Infraestructuras Críticas y guías de buenas prácticas para el sector agua, que se han tenido en cuenta como referencia adicional de cumplimiento, sin ser el foco normativo principal del trabajo.

4. Análisis de amenazas — MITRE ATT&CK for ICS

Se seleccionaron las técnicas de MITRE ATT&CK for ICS³ directamente observables en el laboratorio, evitando enumerar técnicas no demostradas:

¹ IEC 62443 define los niveles de madurez de seguridad (SL1-SL4) y el modelo de zonas y conductos aplicado en la sección 7.2 de este documento.

² Directiva (UE) 2022/2555, artículos 20-21, sobre gestión de riesgos de ciberseguridad y notificación de incidentes.

³ Matriz equivalente a MITRE ATT&CK Enterprise pero orientada específicamente a tácticas y técnicas observadas en entornos de control industrial.

Técnica	Nombre	Fase donde se observa
T0846	Remote System Discovery	Reconocimiento inicial con <code>nmap</code> (sección 6.1)
T0861	Point & Tag Identification	Lectura de coils Modbus con <code>mbpoll</code> (sección 6.2)
T0855	Unauthorized Command Message	Escritura no autorizada en el coil de la válvula (sección 6.3)
T0814	Denial of Control	Discutida como extensión del ataque (sección 6.4, análisis de impacto)

Estas cuatro técnicas cubren el ciclo completo que un atacante seguiría en un escenario real: descubrir la red, identificar qué controla cada registro del PLC, y finalmente manipular el proceso físico. El diseño de detección de la sección 7 se orienta específicamente a T0855, por ser la técnica con mayor impacto físico directo y la explotada en la demostración.

5. Laboratorio y stack tecnológico

5.1 Arquitectura del despliegue

Todo el laboratorio se despliega mediante `docker-compose.yml`, lo que garantiza su reproducibilidad completa con un único comando. El stack está compuesto por:

Servicio	Imagen / origen	Red(es)	Función
<code>openplc</code>	<code>tuttas/openplc_v3</code>	<code>red_ot</code>	PLC simulado, expone Modbus TCP 502
<code>conpot</code>	<code>honeynet/conpot</code>	<code>red_ot</code>	Honeypot ICS (emula un Siemens S7-200)
<code>modbus-sensor</code>	Dockerfile propio (Python + <code>scapy</code>)	comparte interfaz de <code>openplc</code>	Sensor IDS: detecta escrituras Modbus
<code>wazuh.manager</code>	<code>wazuh/wazuh-manager:4.9.0</code>	<code>red_it</code> + <code>red_ot</code>	SIEM: centraliza y clasifica alertas
<code>kali</code>	Dockerfile propio (Debian + <code>nmap/mbpoll/pymodbus</code>)	<code>red_it</code> + <code>red_ot</code>	Atacante simulado

Nota sobre protocolos industriales. Las reglas de detección deben reflejar patrones específicos de los protocolos industriales realmente presentes en el entorno. En este laboratorio solo se implementa y se justifica Modbus, por ser el único protocolo que expone realmente la imagen de OpenPLC utilizada

(tuttas/openplc_v3); Profinet no se implementa porque esa imagen no lo soporta, y añadirlo habría requerido otro controlador simulado sin aportar cobertura real sobre el vector de ataque demostrado.

5.2 Incidencias de reproducibilidad resueltas

Durante el despliegue, y durante las pruebas de reproducibilidad en ordenadores distintos, se detectaron y corrigieron varias incidencias reales, documentadas íntegramente en los anexos de registro de laboratorio:

1. La imagen `thiagoralves/openplc_v3` referenciada inicialmente no existe en Docker Hub; se sustituyó por la imagen pública equivalente `tuttas/openplc_v3`.
2. El contenedor `conpot` no arrancaba al forzar un `command` personalizado, que rompía el `PATH` de instalación del binario; se resolvió respetando el `ENTRYPOINT` por defecto de la imagen.
3. El sensor Modbus requería la librería de sistema `libpcap`, ausente en la imagen base de Python.
4. La configuración de recepción de `syslog` en Wazuh quedó inicialmente insertada dentro de un comentario XML de `ossec.conf` por un error de automatización con `sed`; se corrigió con un script Python que localiza el bloque por contenido, no por número de línea. Posteriormente, al desplegar el Dashboard completo de Wazuh, esta configuración quedó integrada de forma declarativa en `wazuh_manager.conf`, eliminando la necesidad de scripts de parcheo en caliente.
5. La imagen base del contenedor `kali` (`kalilinux/kali-rolling`) resultó poco fiable al reconstruirse en un segundo ordenador, con fallos de resolución de paquetes achacables a sus mirrors; se sustituyó por `debian:bookworm-slim`, con una red de mirrors mucho más redundante.
6. Al probar el laboratorio en un tercer ordenador (Windows con Docker Desktop/WSL2), se detectaron dos incidencias adicionales de entorno, no del código: el reloj interno de la máquina virtual de WSL2 quedó desincronizado varias horas respecto al sistema anfitrión, provocando el rechazo de los repositorios de paquetes por firma “todavía no válida” (se resolvió reiniciando el sistema); y el puerto 8080 estaba ocupado por un servicio ajeno de Windows, lo que impedía publicar el de OpenPLC (se resolvió remapeando el puerto en `docker-compose.yml`, sin tocar el puerto interno del contenedor).

Estas incidencias, lejos de ser un problema, constituyen evidencia de un proceso de despliegue verificado paso a paso — el laboratorio se probó de forma iterativa, en varios sistemas operativos distintos, hasta confirmar cada componente en funcionamiento real, no solo “levantado”.

5.3 Verificación del stack

La figura 2 muestra el panel de control de OpenPLC con el programa de control cargado y en ejecución, tras resolver las incidencias anteriores.

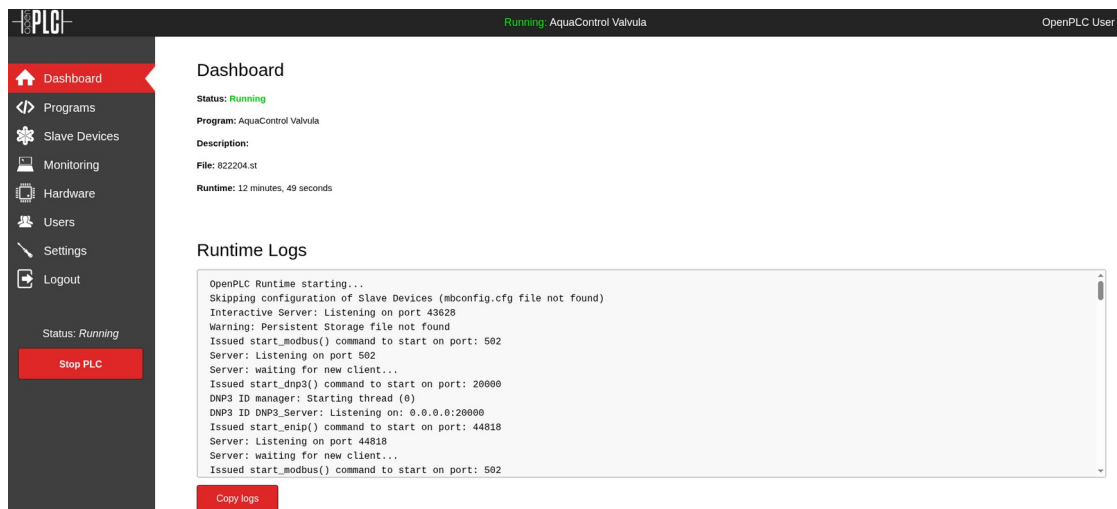


Figura 2. Panel de OpenPLC con el programa AquaControl Valvula en ejecucion.

El programa de control cargado (`aquacontrol_valvula.st`, anexo) implementa una lógica mínima pero representativa: una válvula y una bomba se activan automáticamente cuando un sensor de nivel de tanque indica nivel bajo, y se desactivan en caso contrario. Esta lógica activa es la que permite, en la sección 6.4, observar un hallazgo relevante sobre los límites de un ataque de escritura puntual.

6. Demostración de ataques

6.1 Reconocimiento de la red OT (T0846)

Desde el contenedor `kali`, conectado a ambas redes representando un atacante que ya ha cruzado el perímetro:

```
docker exec -it kali nmap -sV 172.21.0.0/24
```

El escaneo identificó los tres hosts de la red OT. Un hallazgo destacable: al recibir una petición HTTP no reconocida, Conpot respondió con una página de error que se identifica explícitamente como un controlador **Siemens SIMATIC S7-200 (CP 443-1 EX40)**, confirmando que el honeypot emula correctamente un dispositivo industrial real ante un escaneo externo.

En este primer escaneo, el puerto Modbus (502) de OpenPLC **no** apareció abierto, al no estar todavía activado el servidor Modbus ni cargado el programa de control — un hallazgo metodológico en sí mismo: la superficie de ataque de un PLC depende de su estado operativo, no solo de la imagen desplegada.

6.2 Identificación de registros (T0861)

Tras activar el servidor Modbus y cargar el programa de control, una nueva verificación confirmó el puerto 502 abierto (`502/tcp open modbus Modbus TCP`). Se procedió a leer los coils del PLC:

```
docker exec -it kali mbpoll -m tcp -t 0 -c 3 -r 1 -1 172.21.0.10
```

Resultado: los tres coils consultados (`Valvula1`, `Bomba1` y un margen) devolvieron valor `0`, coherente con la ausencia de un sensor de nivel activo.

6.3 Escritura no autorizada (T0855)

Sin ningún tipo de autenticación, se forzó el coil de `Valvula1` a `1` (abierta):

```
docker exec -it kali mbpoll -m tcp -t 0 -r 1 172.21.0.10 1
```

El PLC confirmó la escritura (`Written 1 references.`). Este es el vector de ataque central del TFM: cualquier host con conectividad de red al puerto 502 puede alterar el estado de un actuador físico sin ninguna verificación de identidad.

6.4 Análisis de impacto

Al leer de nuevo el coil inmediatamente después de la escritura, `Valvula1` apareció otra vez en `0`. La causa: el programa de control se reevalúa cada 100 ms, y su lógica (`IF NivelBajo THEN Valvula1 := TRUE ELSE Valvula1 := FALSE`) revierte automáticamente el valor forzado en el siguiente ciclo de *scan*, al seguir `NivelBajo` en `0`.

Este resultado, lejos de ser un fallo de la demostración, constituye un hallazgo relevante: **un ataque de escritura puntual contra un PLC con lógica de control activa tiene impacto limitado**, porque el propio programa revierte el estado forzado — aunque la vulnerabilidad de fondo (aceptar la orden sin autenticación) sí se explota con éxito en todos los casos. Para lograr un efecto sostenido, un atacante necesitaría sostener la escritura en bucle a una frecuencia mayor que el ciclo de *scan* del PLC (variante combinable con T0814 — Denial of Control), o comprometer directamente el valor leído del sensor en origen. Esta conclusión justifica, en la sección 7, por qué la **detección temprana** es más determinante que confiar en que la lógica del PLC “se autoprotege”.

7. Medidas de protección

7.1 Detección: sensor Modbus y reglas Wazuh

Se descartó instalar un agente Wazuh dentro de la imagen de OpenPLC (de terceros), por la fragilidad de reproducibilidad que introduce el enrolamiento de claves. En su lugar se implementó un **sensor propio** (`modbus-sensor`, Python + `scapy`) que comparte la interfaz de red del contenedor `openplc` — evitando así la limitación de un *bridge* de Docker, que no reenvía a un tercer contenedor tráfico unicast ya aprendido entre otros dos.

El sensor inspecciona el tráfico al puerto 502 y, al detectar un código de función de **escritura** (5, 6, 15 o 16), envía una alerta por *syslog* UDP a Wazuh. Tras resolver la configuración de recepción de *syslog* en `ossec.conf` y desplegar una regla local personalizada (`aquacontrol_rules.xml`, nivel 10, grupo `modbus,ics,aquacontrol`), se repitió el ataque de la sección 6.3 y se confirmó la alerta:

```
Rule: 100050 (level 10) -> 'AquaControl: escritura Modbus no
autorizada detectada (posible manipulacion de valvula/bomba via PLC) -
T0855'
ModbusIDS:      Unauthorized      write      detected      src=172.21.0.50
function_code=5 function_name="Write Single Coil" unit_id=1 address=0
value=65280
```

Con esto queda demostrado el ciclo completo: **reconocimiento** → **ataque** → **detección**, con la alerta correlacionando correctamente el origen real del atacante (172.21.0.50) incluso cuando el sensor observa el tráfico desde la interfaz de red de OpenPLC.

La figura 3 muestra la alerta ya clasificada en el Dashboard de Wazuh, y la figura 4 el detalle completo del evento (incluyendo el log crudo con la IP real del atacante y el grupo de reglas disparado).

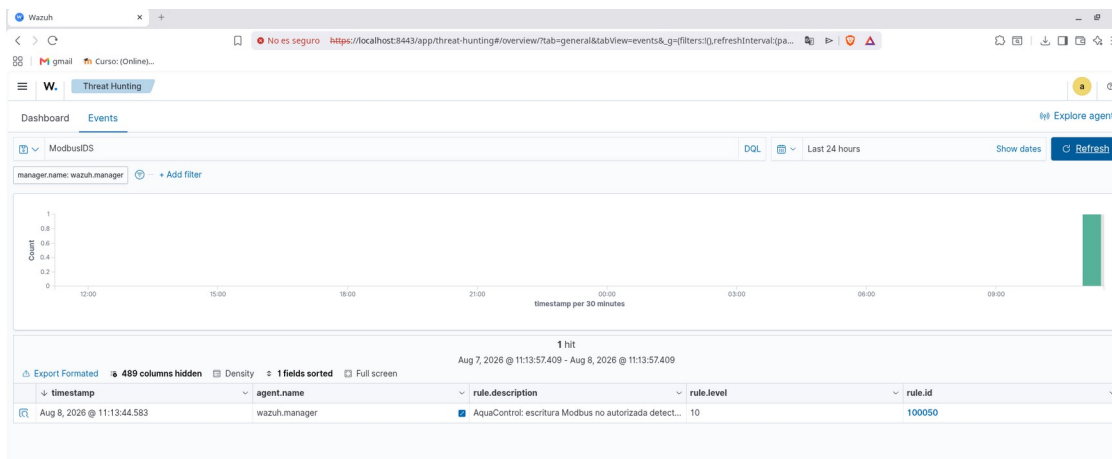


Figura 3. Alerta AquaControl (regla 100050) en el Dashboard de Wazuh.

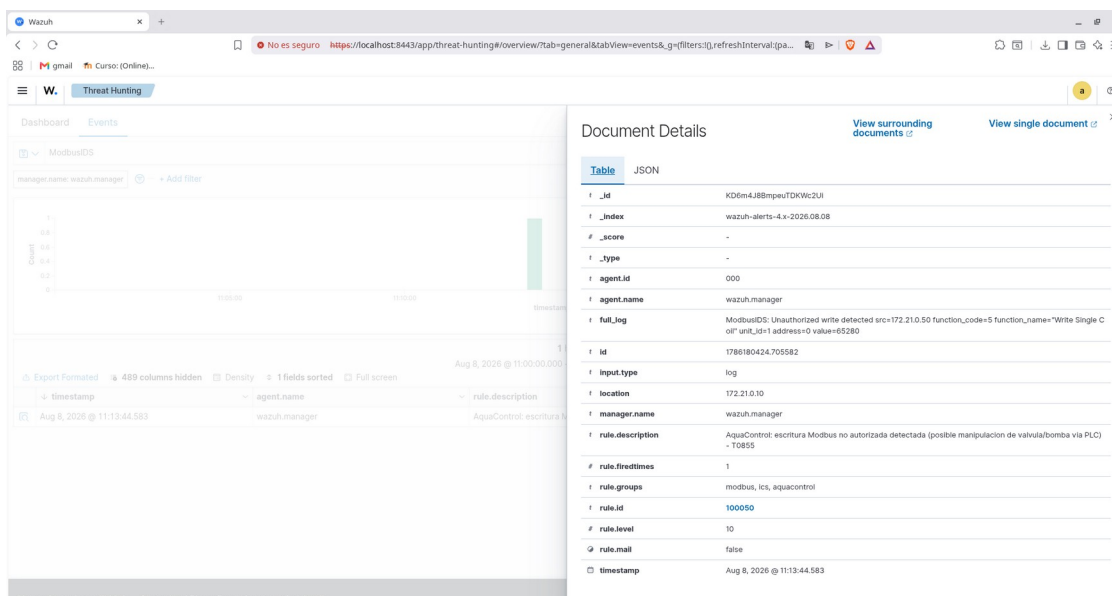


Figura 4. Detalle completo del evento: full_log, rule.description, rule.groups y rule.level.

7.2 Corrección: segmentación real y conducto controlado

La figura 5 muestra la arquitectura corregida propuesta, con una DMZ intermedia y las zonas IEC 62443 aplicadas a la red OT.

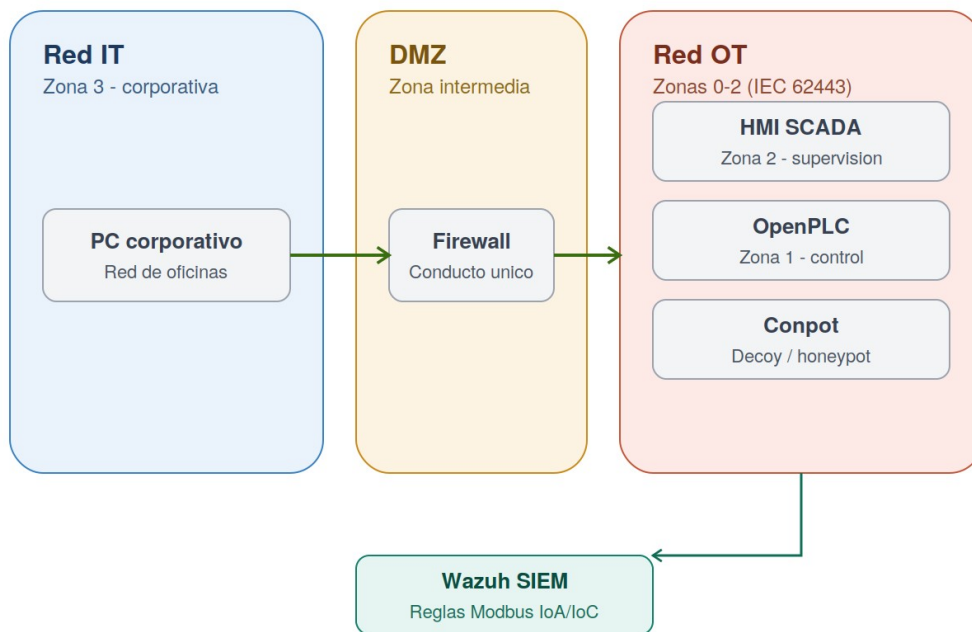


Figura 5. Arquitectura corregida: DMZ y zonas IEC 62443.

Esta arquitectura no se dejó únicamente como propuesta teórica: se implementó y verificó en un **segundo laboratorio Docker independiente**. Se evaluaron dos aproximaciones para materializar el “conducto único controlado”: un firewall con enrutamiento a nivel de kernel (`iptables/ip_forward`) y un **proxy de aplicación** (`socat`). Se optó por la segunda, al no requerir configuración de enrutamiento del host —origen habitual de laboratorios no reproducibles en otro ordenador— y bastar con un contenedor dual-homed que reenvía en espacio de usuario únicamente el puerto de gestión web autorizado (8080), sin reenviar en ningún caso el puerto Modbus (502).

En este segundo laboratorio, Kali ya no está conectado a la red OT (representa un atacante confinado a la red IT tras la corrección). Los resultados de las tres pruebas de verificación fueron:

Prueba	Resultado	Interpretación
Kali → OpenPLC directo (502, 8080)	<code>filtered</code> (sin respuesta)	Sin ruta de red entre IT y OT: la segmentación es efectiva
Kali → conducto (8080, gestión web)	<code>HTTP/1.0 302 FOUND</code>	El mantenimiento remoto legítimo sigue funcionando
Kali → conducto (502, Modbus)	<code>Connection refused</code>	El vector de ataque de la sección 6.3 queda cerrado

Las figuras 6 a 8 muestran la salida real de cada una de las tres pruebas.

```

~/Escritorio/TFM-AquaControl/laboratorio-corregido main*
> docker exec -it kali-corregido nmap -p 502,8080 -Pn 172.31.0.10
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-08 09:20 +0000
Nmap scan report for 172.31.0.10
Host is up.

PORT      STATE      SERVICE
502/tcp    filtered   mbap
8080/tcp    filtered   http-proxy

Nmap done: 1 IP address (1 host up) scanned in 7.07 seconds

```

Figura 6. Prueba 1 - acceso directo Kali/OpenPLC filtrado.

```

~/Escritorio/TFM-AquaControl/laboratorio-corregido main* 7s
> docker exec -it kali-corregido curl -i http://172.30.0.20:8080
HTTP/1.0 302 FOUND
Content-Type: text/html; charset=utf-8
Content-Length: 219
Location: http://172.30.0.20:8080/login
Vary: Cookie
Set-Cookie: session=eyJfZnJlc2giOmZhbnVLLCJfcGVybWZuZS0Ijp0cnVlfQ.anb1KA.3Lv8BnpPb0_OqLhYcmx1hoLmWwY; Expires=Sat, 08-Aug-2026 09:26:44 GMT; HttpOnly; Path=/
Server: Werkzeug/0.16.0 Python/2.7.17
Date: Sat, 08 Aug 2026 09:21:44 GMT

```

Figura 7. Prueba 2 - acceso a gestion web via el conducto (302 FOUND).

```

~/Escritorio/TFM-AquaControl/laboratorio-corregido main*
> docker exec -it kali-corregido nc -zv -w 3 172.30.0.20 502
nc: connect to 172.30.0.20 port 502 (tcp) failed: Connection refused

~/Escritorio/TFM-AquaControl/laboratorio-corregido main*
>

```

Figura 8. Prueba 3 - Modbus bloqueado incluso via el conducto (Connection refused).

Esta tabla demuestra empíricamente —no solo en diagrama— que la corrección propuesta cumple su objetivo sin afectar la operación legítima, alineándose con el requisito de IEC 62443 de mantener conductos controlados en lugar de una segmentación que bloquee también el tráfico autorizado.

8. Conclusiones

Este TFM ha recorrido el ciclo completo de un ejercicio de seguridad ofensiva-defensiva sobre un escenario ICS/SCADA deliberadamente imperfecto: documentación de deficiencias, explotación controlada de un vector real (Modbus sin autenticación), detección mediante un sensor e integración SIEM propios, y corrección verificada empíricamente en un entorno independiente.

Lecciones aprendidas. La superficie de ataque de un sistema ICS depende de su estado operativo (un PLC sin programa cargado no expone su protocolo de control). Un ataque de escritura puntual contra un PLC con lógica activa tiene impacto limitado por autocorrección del propio ciclo de control, lo que resalta la importancia de la detección temprana frente a la confianza en la robustez incidental del proceso controlado. La segmentación de red mediante conductos de aplicación (en lugar de enrutamiento a nivel de kernel) resulta más reproducible y igualmente efectiva para los fines de este laboratorio.

Limitaciones. El laboratorio usa una lógica de control deliberadamente simplificada (una válvula y una bomba); un proceso real tendría más variables de estado y, por tanto, mayor superficie para ataques sostenidos. El sensor Modbus es un IDS de firma simple (por código de función), no un motor de análisis

de protocolo con estado. La arquitectura corregida no se probó bajo carga ni con múltiples clientes legítimos concurrentes.

Propuestas de mejora escalables. Añadir detección de anomalías basada en el valor escrito (no solo el código de función), extender el sensor a Conpot para correlacionar honeypot + PLC real, y evaluar el uso de un *data diode* físico como alternativa de máxima garantía al conducto de aplicación en un despliegue de producción real.

9. Bibliografía

1. International Electrotechnical Commission. *IEC 62443 — Industrial communication networks — Network and system security*. <https://www.iec.ch/homepage>
2. Parlamento Europeo y Consejo. *Directiva (UE) 2022/2555 (NIS2)*. <https://eur-lex.europa.eu>
3. Gobierno de España. *Real Decreto 311/2022, Esquema Nacional de Seguridad*. CCN-CERT. <https://www.ccn-cert.cni.es>
4. Centro Nacional para la Protección de las Infraestructuras Críticas (CNPIC). <https://www.cnpic.es>
5. MITRE. *ATT&CK for ICS*. <https://attack.mitre.org/matrices/ics/>
6. Autonomy Logic. *OpenPLC Runtime — Documentación oficial*. <https://autonomylogic.com>
7. The Honeynet Project. *Conpot — ICS/SCADA Honeypot*. <https://github.com/mushorg/conpot>
8. Wazuh Inc. *Wazuh Documentation*. <https://documentation.wazuh.com>
9. Docker Inc. *Docker Compose Documentation*. <https://docs.docker.com/compose/>
10. pymodbus contributors. *pymodbus — Python Modbus library*. <https://pymodbus.readthedocs.io>
11. Langner, R. *Stuxnet: Dissecting a Cyberwarfare Weapon*. IEEE Security & Privacy, 2011.
12. Prensa especializada. *Caso Oldsmar, Florida (2021): acceso remoto no autorizado a sistema de tratamiento de agua*.
13. ESET / Dragos. *Industroyer/CRASHOVERRIDE: análisis del ataque a la red eléctrica ucraniana (2016)*.
14. ENISA. *Threat Landscape for Industrial Automation Systems (informe anual)*. <https://www.enisa.europa.eu>